

First Day at the Command Line

My live demo for the first day at the terminal

This was my own plan for the Linux intro, not really meant as a handout for students. Because we got behind and the recording did not work out, I am giving you these notes. I pulled the sequence together from common recommendations I found online for teaching the command line, then arranged it the way I like to work through it in class. Each step gives me something to say, a command to run, and the thing to point out once it runs. In going over this on, we got behind and jump around, and I was not able to get to all. I encourage you to practice these on your own.

A note before you begin. I assembled these notes myself from online resources and I revise them often, so an occasional error or typo can slip through. If something here does not behave the way I describe when you try it, please tell me.

Who is on what. I am running this demo on Arch Linux, using Alacritty as my terminal. If you are a student reading this by accident, that is fine: you are most likely on Void (our lab) or on the XUbuntu virtual machine, and every command below behaves the same on all three. The only things that differ between systems are how you open a terminal and how you install software, and both are shown for each system where they come up.

1. Open a terminal

How you open a terminal depends on the machine you are sitting at.

- **On my machine (Arch):** I launch **Alacritty**, from the app launcher or its keyboard shortcut.
- **On the XUbuntu VM:** open the **Applications** menu and choose **Terminal Emulator**, or press **Ctrl+Alt+T** if the shortcut is enabled.
- **On Void:** open the terminal your desktop provides from its menu; the exact name varies by setup.

Whichever one you use, a window appears with a blinking cursor after a short line of text. That line is the **prompt**, and it is waiting for you to type.

Say out loud what the prompt is telling you: your user name, the machine name, and where you currently are in the filesystem. On my screen the host reads **arch**; on the student machines it will read **void** or **xubuntu** instead, but the shape is the same.

```
you@machine:~$  
# user      host      location  waiting-for-you
```

2. Who and where am I

Start with three harmless commands that answer three honest questions.

```
whoami
hostname
pwd
```

Point out that `pwd` stands for *print working directory*, and that the answer is almost certainly `/home/student`, which the prompt was already abbreviating as the tilde character.

3. Look around

Now show the room what is here.

```
ls
ls -l
ls -a
ls -lh
```

Explain the pattern as you go: `ls` lists names, and the pieces after it that begin with a dash are **options** that change the behavior. The `-l` option gives one item per line with details, `-a` reveals the hidden dotfiles, and `-h` makes the sizes human readable. Note that options can be combined, which is why `-lh` works.

4. Move around

Walk into a folder, look, then walk back out.

```
cd /
ls
cd /home
ls
cd ~
pwd
cd ..
pwd
```

This is the moment to draw the filesystem as a tree on the board. Explain that `/` is the root of everything, that `~` is a shortcut for your home directory, and that `..` always means the folder one level up. Make the distinction between an **absolute** path, which starts at `/`, and a **relative** path, which starts from wherever you happen to be.

5. Make something

Create a place to work, then create a file inside it.

```
cd ~
mkdir demo
cd demo
touch notes.txt
ls
```

Point out that `mkdir` makes a directory and `touch` creates an empty file. The file is real now, even though it holds nothing yet.

6. Put text into a file

Show that the command line can write as well as read.

```
echo "hello from the terminal" > notes.txt
cat notes.txt
echo "a second line" >> notes.txt
cat notes.txt
```

This is the first genuinely surprising moment for most beginners, so slow down. A single angle bracket, `>`, sends the output into the file and replaces whatever was there. Two angle brackets, `>>`, add to the end instead. `cat` prints the file back so the class can see the difference.

7. Edit a file with vim

This is the editor we will lean on most this term, so it is worth a careful first look. Open the file you just made.

```
vim notes.txt
```

Warn the class before anyone touches a key: vim starts in **Normal mode**, where the keys are commands rather than text. Press `i` to switch to **Insert mode**, then type a line or two so they see it behave like an ordinary editor. Press `Esc` to return to Normal mode, and narrate the switch each way so the idea of modes lands.

Now show how to leave, which is the part everyone remembers.

```
:w      # save the file
:wq     # save and quit
:q!     # quit without saving
```

Make the escape hatch explicit and have the whole room do it once: press `Esc`, then type `:q!` and press Enter. Say plainly that if they ever feel trapped in vim, this is the way out, and that needing it is completely normal. Mention that `nano` is also installed as a simpler alternative, which we will see in the written guide.

8. Copy, rename, and remove

```
cp notes.txt backup.txt
ls
mv backup.txt archive.txt
ls
rm archive.txt
ls
```

Explain that `cp` copies, `mv` both moves and renames depending on where you point it, and `rm` deletes. Pause on `rm` and say plainly that there is no recycle bin here. A deleted file is gone. This is the one command where reading twice before pressing Enter is a habit worth building on the first day.

9. Read longer things

```
ls /usr/bin | less
```

This pipes a long list into a pager. Show them how to scroll, then how to leave: press `q` to quit. Mention `head` and `tail` as the two commands that show only the first or last few lines of a file, which is handy when a file is large.

10. Ask the system for help

No one memorizes these commands. They look them up.

```
man ls
```

The manual opens in the same pager, so `q` quits again. Then show the quicker forms.

```
ls --help
```

Mention that many systems also have `tlldr`, which prints a handful of common examples instead of the full manual, and that it is often the friendlier starting point.

11. A first taste of power

Combine two commands so the output of one becomes the input of the next.

```
ls /usr/bin | wc -l  
ls /usr/bin | grep python
```

The vertical bar is a **pipe**. The first line counts how many programs live in that folder, and the second keeps only the lines that mention Python. Tell them this idea, small tools joined by pipes, is the whole philosophy of the command line in one sentence.

12. Housekeeping and shortcuts

End with the comforts that make daily work bearable.

```
history  
clear
```

Demonstrate three things by hand rather than by command: press the **Up arrow** to bring back the previous command, type the first few letters of a name and press **Tab** to complete it, and press **Ctrl+C** to cancel a command that is running or that you started by mistake.

13. Installing software across our distros

You do not need students to install anything today, but it is worth showing that the idea is universal and only the spelling changes. Each distribution has a **package manager**.

Table 1: Package manager commands by distribution.

Distribution	Install a program	Update everything
XUbuntu (students)	<code>sudo apt install NAME</code>	<code>sudo apt update</code>
Arch (my machine)	<code>sudo pacman -S NAME</code>	<code>sudo pacman -Syu</code>
Void	<code>sudo xbps-install NAME</code>	<code>sudo xbps-install -Su</code>

The takeaway is not the exact flags. It is that every system has one blessed way to add and update software, and that reaching for it is safer than downloading programs by hand.

14. Wrap up

Close the file, close the folder, and close the terminal.

```
cd ~  
exit
```

If anything goes sideways during the demo, the recovery lines are short and worth saying out loud so students do not panic later: a *command not found* message almost always means a typo, **Ctrl+C** stops a runaway command, and pressing the Up arrow brings back what you just typed so you can fix one character instead of retyping the whole line.